

残差块 BasicBlock 逐行解析

对应源码: `models/backbone.py` 第 13-33 行

对应论文: ResNet (He et al., CVPR 2016, ImageNet 冠军)

学习目标: 理解 ResNet 为什么能堆到上百层还能训练

一、它要解决什么问题

朴素网络的困境

朴素卷积网络结构:

```
输入 x → conv1 → relu → conv2 → relu → 输出
```

理论上网络越深越强, 但实际堆到 20 层以上, 效果反而变差。

这不是过拟合, 而是训练时梯度传不回浅层, 浅层学不动。

这个现象叫做 网络退化 (Degradation) 。

残差块的核心思想

不让网络"重新学一个输出", 而是让它学"在输入基础上要改多少":

- 朴素网络: 输出 = $F(x)$ (从零学出完整答案)
- 残差网络: 输出 = $F(x) + x$ (只学修正量, 原信息走捷径保留)

那个 $+ x$ 就是 残差连接 (shortcut / skip connection) 。

一个类比: 改作文

- 朴素网络 = 老师让你把整篇文章重写一遍 (容易越改越糟)
 - 残差网络 =
老师让你在原稿上用红笔批改 (原稿保留, 只动需要改的地方, 最差也不会比原稿烂)
-

二、完整源码 (21 行)

```

class BasicBlock(nn.Module):
    """简化的 ResNet BasicBlock (无 BottleNeck, 适合小模型)"""

    def __init__(self, in_ch: int, out_ch: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.downsample = nn.Identity()
        if stride != 1 or in_ch != out_ch:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                nn.BatchNorm2d(out_ch),
            )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = self.downsample(x)
        out = F.relu(self.bn1(self.conv1(x)), inplace=True)
        out = self.bn2(self.conv2(out))
        return F.relu(out + identity, inplace=True)

```

三、构造函数逐行讲解

第 16 行: 三个参数

```
def __init__(self, in_ch, out_ch, stride=1):
```

- **in_ch**: 输入通道数 (例如 32)
- **out_ch**: 输出通道数 (例如 64)
- **stride**: 步长。=2 时图像缩小一半 (降采样) ; =1 时尺寸不变

第 18-19 行: 第一层卷积 + BN

```

self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
self.bn1 = nn.BatchNorm2d(out_ch)

```

- **Conv2d(..., 3, stride, 1)**: 3×3 卷积窗口; **stride** 控制是否缩小;
最后的 1 是 padding=1, 保证 3×3 卷积后尺寸不额外变化
- **bias=False**: 后面紧跟 BN, bias 会被 BN 抵消, 写了也白写, 去掉省参数
- **BatchNorm (BN)**: 把每个通道的数据拉回均值 0、方差 1 附近。

作用是让训练更稳、可用更大学习率、收敛更快。

可以理解为给数据做"标准化体检", 防止数值越滚越大或越滚越小

第 20-21 行: 第二层卷积 + BN

```
self.conv2 = nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
self.bn2 = nn.BatchNorm2d(out_ch)
```

- 这层 `stride=1`，不再缩小
- 设计逻辑：降采样只在 `conv1` 做一次，`conv2` 专心做特征变换
- 两层 3×3 叠加，感受野等于一层 5×5 ，但参数更少、非线性更强

第 22-27 行：shortcut 分支（重点）

```
self.downsample = nn.Identity()
if stride != 1 or in_ch != out_ch:
    self.downsample = nn.Sequential(
        nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
        nn.BatchNorm2d(out_ch),
    )
```

问题：最后要做 `out + identity`，两边形状必须一模一样才能相加。但：

- 如果 `stride=2`，`out` 的空间尺寸是 `x` 的一半 → 对不上
- 如果 `in_ch=32`，`out_ch=64`，`out` 有 64 通道，`x` 只有 32 → 对不上

所以需要有一个“形状适配器”把 `x` 也变成 `out` 的形状：

- `nn.Identity()`：形状本来就匹配时，原样返回，什么都不做
- 1×1 卷积：形状不匹配时，用 1×1 小卷积同时完成

“调通道数 + 跟着缩小尺寸”。 1×1 卷积不看邻居，只做通道的线性混合，非常廉价

类比：`out` 和 `x` 要并排合影，但身高不同。

`downsample` 就是给矮的那个垫个箱子，让两人一样高。

四、forward 数据流逐行讲解

```
def forward(self, x):
    identity = self.downsample(x)          # ① 先把“原稿 x”复印一份收好
    out = F.relu(self.bn1(self.conv1(x)))  # ② 主路：conv1 → BN → ReLU
    out = self.bn2(self.conv2(out))        # ③ 主路：conv2 → BN（先不激活）
    return F.relu(out + identity)          # ④ 原稿 + 批改 → 再 ReLU
```

① `identity = downsample(x)`

把原始输入存起来（可能垫了箱子调形状）。这就是那条“捷径”。

② `conv1 → bn1 → relu`

第一次特征提取 + 标准化 + 非线性激活。

ReLU 的作用：负数清零，正数保留。引入非线性，网络才能学复杂曲线。

没有它，堆多少卷积都等价于一个大线性变换。

③ conv2 → bn2 (注意没有 ReLU)

第二次特征提取 + 标准化。

这里故意不接 ReLU，因为马上要和 identity 相加。

如果先 ReLU 把信息砍成非负，可能破坏 identity 里的负信息。

④ out + identity, 然后才 ReLU

主路学到的"修正量" + 原始输入，合并后再统一激活。

这是整个残差块的灵魂。

五、数据流形状实例

以 stage2 的一个 block 为例 (in_ch=32, out_ch=64, stride=2) ,

输入形状 [B, 32, 64, 64]:



两条路最终都是 [B,64,32,32]，可以安全相加。

六、为什么这个设计这么关键 (梯度原理)

反向传播时，损失的梯度可以沿 + identity 这条捷径

直接、无衰减地流回浅层：

梯度流到 "+" 节点时分成两股：

$$d(\text{out} + \text{identity}) / dx = d(\text{out})/dx + 1$$

这个 "1" 保证梯度至少能原样传一份回去

- 朴素网络：梯度要连乘很多层权重，越乘越小 (梯度消失)

- 残差网络：多了一个常数 1 的通路，梯度永远有一条高速公路直达浅层
- 所以 ResNet 堆 50 层、101 层甚至 152 层都能训动。

七、5 个设计决策速记表

代码	为什么这么设计
两个 3×3 卷积	等效一个 5×5 感受野，但参数更少、多一层非线性
conv 后接 BN	稳定数值分布，训练更快更稳，可用大学习率
conv 的 bias=False	BN 会抵消 bias，省掉无用参数
downsample 的 1×1 conv	只在通道/尺寸不匹配时"垫箱子"，廉价对齐形状
relu(out + identity)	残差连接：梯度有 +1 高速路，深层网络可训练

八、一句话记忆

残差块不让网络重写作文，只让它在原稿上批改——
原稿永远保留，所以再深也不会改烂。

九、自测问题（学完检查）

1. 为什么 conv2 后面、相加之前不接 ReLU？
2. 什么情况下 downsample 是 Identity，什么情况下是 1×1 卷积？
3. 残差连接为什么能缓解梯度消失？（提示：那个 +1）
4. 两个 3×3 卷积叠加，等效多大的感受野？
5. 为什么卷积层设置 bias=False？

参考答案

1. 因为马上要和 identity 相加，先 ReLU 会把信息砍成非负，可能破坏 identity 里的负信息。
2. stride=1 且 in_ch=out_ch 时形状匹配，用 Identity；否则用 1×1 卷积对齐形状。
3. 反传时 $d(out+x)/dx = d(out)/dx + 1$ ，常数 1 保证梯度至少原样传回一份。
4. 等效 5×5 感受野。
5. 后面紧跟 BatchNorm，bias 会被 BN 的归一化抵消，属于无用参数。